

ВСТУП

Стрімкий розвиток інформаційних технологій та глобалізація мережі Інтернет докорінно змінили парадигму створення та розповсюдження медіаконтенту. На зміну централізованим моделям прийшли розподілені мережі, де кожен користувач може бути генератором інформації. Ця демократизація, попри переваги свободи слова, створила сприятливе підґрунтя для безконтрольного поширення дезінформації, що набуває ознак глобальної загрози.

У межах даної курсової роботи розглядається проектування та програмна реалізація системи VERITAS — браузерного розширення для Google Chrome, призначеного для автоматичного аналізу, верифікації та оцінки достовірності новинного медіаконтенту. Система використовує гібридний підхід: поєднання семантичного аналізу тексту за допомогою великих мовних моделей (LLM) та детермінованих математичних алгоритмів для формування об'єктивного рейтингу довіри.

Актуальність теми зумовлена зростаючою загрозою інформаційних маніпуляцій, недостатньою ефективністю існуючих рішень, що базуються на статичних списках доменів, та появою нових можливостей автоматизованого контент-аналізу завдяки розвитку трансформерних моделей.

Об'єкт дослідження — процес аналізу та верифікації новинного медіаконтенту в цифровому середовищі.

Предмет дослідження — методи обробки природної мови та математичні алгоритми оцінювання, що застосовуються для автоматичної перевірки достовірності новинних статей.

Мета дослідження — розробити систему аналізу та верифікації новинного медіаконтенту, яка на основі мікроагентної архітектури штучного інтелекту та авторського алгоритму WMFA (Weighted Multi-Factor Analysis) здатна визначати рівень достовірності інформації, перевіряти факти через OSINT-джерела та формувати зважену оцінку якості новинного матеріалу.

Структура роботи. Курсова робота складається зі вступу, трьох розділів, висновків та списку використаних джерел. У першому розділі розглядаються теоретичні основи проблеми та обґрунтовується вибір концепції системи. У другому розділі наведено опис архітектури, алгоритмів та програмної реалізації. Третій розділ присвячено тестуванню та аналізу результатів.

РОЗДІЛ 1 АНАЛІЗ ПРОБЛЕМИ ТА МЕТОДІВ ВЕРИФІКАЦІЇ МЕДІАКОНТЕНТУ

1.1 Проблематика поширення недостовірного контенту в онлайн-середовищі

Стрімкий розвиток інформаційних технологій та демократизація процесів створення контенту спричинили безпрецедентне зростання обсягу інформації в мережі Інтернет. Водночас ця свобода створила сприятливе підґрунтя для безконтрольного поширення дезінформації. Основною проблемою сучасного інформаційного простору є еволюція методів маніпулювання: якщо раніше загрозою були прямі фальсифікації, то сьогодні на перший план виходять складні маніпуляції, що базуються на реальних фактах, але використовують виривання з контексту, зміщення акцентів та емоційну лексику для формування хибного сприйняття [1].

Ситуацію ускладнюють алгоритми соціальних мереж, що працюють за принципами «економіки уваги» (attention economy). Пріоритет надається емоційно забарвленому контенту, який поширюється значно швидше за об'єктивну аналітику [2]. Це стимулює авторів використовувати сенсаційні заголовки та провокаційний зміст, розмиваючи межі між журналістикою та пропагандою. Додатковим викликом стала доступність генеративного штучного інтелекту, що дозволяє автоматизувати створення переконливих фейків у масштабах, які унеможливлюють ручну модерацію.

З боку користувача проблема посилюється інформаційним перевантаженням та поверхневим споживанням контенту. Критичний аналіз кожного повідомлення вимагає часу та переходу на сторонні ресурси перевірки, що є суттєвим бар'єром для більшості людей. Як наслідок, виникає потреба в автоматизованих інструментах, здатних інтегруватися безпосередньо у процес споживання новин.

1.2 Огляд існуючих підходів та програмних засобів

Існуючі системи виявлення недостовірного контенту можна класифікувати за підходом до аналізу. Перша категорія — репутаційні сервіси, такі як NewsGuard та Web of Trust (WOT), що формують рейтинги на основі ручного аудиту домену чи краудсорсингу. Їхня перевага — висока точність для відомих джерел, але ключовим обмеженням є статичність: вони не аналізують конкретну статтю, а лише присвоюють усім матеріалам сайту однакову оцінку [3].

Друга категорія — фактчекінгові організації (StopFake, VoxCheck, Snopes), що проводять ручну перевірку окремих тверджень. Такий підхід забезпечує глибину

аналізу, проте має критичне обмеження за швидкістю: перевірка однієї статті займає від кількох годин до кількох днів, що робить неможливим охоплення потоку інформації в реальному часі [4].

Третя категорія — автоматизовані класифікатори на основі машинного навчання. Вони використовують лінгвістичні ознаки тексту (частотність слів, стилістичні маркери) для класифікації матеріалу як «fake» або «real». Однак такі бінарні системи не враховують спектральну природу достовірності: стаття може містити переважно правдиву інформацію з лише одним маніпулятивним твердженням.

Жоден з описаних підходів не реалізує одночасно: (а) аналіз конкретного тексту, (б) верифікацію окремих фактів через зовнішні джерела, та (в) формування кількісної зваженої оцінки. Саме ця ніша визначає актуальність розробки системи VERITAS.

1.3 Технологічна основа: NLP, трансформерні моделі та Retrieval-Augmented Generation

Фундаментом для побудови системи верифікації нового покоління є сучасні методи обробки природної мови (NLP). Архітектура трансформерів, запропонована у роботі [5], завдяки механізму self-attention дозволяє аналізувати контекст речення цілісно, а не послідовно. Це уможливлює виявлення складних маніпулятивних патернів, таких як підміна понять чи емоційне фреймування.

Великі мовні моделі (Large Language Models, LLM), побудовані на трансформерній архітектурі, здатні виконувати задачі класифікації стилю тексту, виявлення суб'єктивних суджень та вилучення іменованих сутностей (Named Entity Recognition, NER) без додаткового навчання — шляхом промпт-інжинірингу [6]. Це дозволяє використовувати єдину модель для множини аналітичних задач.

Для верифікації фактів ключовою технологією є Retrieval-Augmented Generation (RAG) [7] — метод, що поєднує генеративну мовну модель із пошуком у зовнішніх джерелах. У контексті фактчекінгу RAG дозволяє: (а) формувати пошукові запити на основі тверджень зі статті; (б) отримувати релевантні фрагменти з авторитетних джерел; (в) семантично порівнювати твердження з отриманими доказами, визначаючи тип зв'язку — підтвердження (Entailment), спростування (Contradiction) або нейтральність [8].

Саме комбінація LLM для стилістичного аналізу та RAG для фактичної верифікації створює технологічну основу для побудови гібридної системи, здатної проводити багатовимірний аналіз контенту.

1.4 Обґрунтування концептуальної моделі системи

На основі проведеного аналізу пропонується побудувати систему верифікації медіаконтенту, що базується на трьох ключових принципах:

- а) контентно-орієнтований аналіз — оцінюється конкретний текст статті, а не репутація домену;
- б) багатофакторна зважена оцінка — замість бінарної класифікації («fake»/«real») формується числовий рейтинг довіри на шкалі від 0 до 100, що відображає комплексну якість матеріалу;
- в) гібридна верифікація — поєднання AI-аналізу стилю тексту з автономною перевіркою фактів через зовнішні OSINT-джерела.

Архітектурно систему доцільно реалізувати як клієнт-серверне рішення у формі браузерного розширення. Клієнтська частина інтегрується безпосередньо у браузер користувача та відповідає за вилучення тексту, візуалізацію результатів та підсвічування сумнівних фрагментів. Серверна частина виконує ресурсоємні обчислення: мультиагентний AI-аналіз, OSINT-верифікацію та детерміноване скорингове обчислення.

Такий підхід дозволяє уникнути поверхневості репутаційних систем та забезпечити глибину аналізу, наближену до ручного фактчекінгу, але в автоматичному режимі та з прозорою кількісною оцінкою.

РОЗДІЛ 2 ПРОЄКТУВАННЯ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ VERITAS

2.1 Архітектура системи та вибір технологічного стеку

Система VERITAS побудована за клієнт-серверною архітектурою з чітким розподілом відповідальностей між компонентами. Клієнтську частину реалізовано як браузерне розширення для Google Chrome, серверну — як REST API на базі фреймворку FastAPI.

Обґрунтування вибору клієнт-серверної архітектури зумовлено кількома чинниками. По-перше, AI-аналіз вимагає взаємодії з зовнішніми API великих мовних моделей, що потребує захищеного серверного середовища для зберігання ключів доступу. По-друге, детерміновані обчислення скорингового алгоритму виконуються на сервері мовою Python, що забезпечує точність математичних обчислень та можливість модульного тестування. По-третє, серверна архітектура дозволяє

оновлювати логіку аналізу без необхідності перевстановлення розширення користувачем.

Для реалізації серверної частини обрано такий технологічний стек:

- а) FastAPI (Python) — асинхронний веб-фреймворк з автоматичною валідацією даних через Pydantic та вбудованою документацією OpenAPI. Асинхронність дозволяє ефективно обробляти конкурентні запити до зовнішніх API [9];
- б) Google Gemini API — сімейство великих мовних моделей (gemini-2.5-flash як основна модель), що забезпечують високу якість аналізу тексту при низькій латентності. Використовується через офіційний SDK google-generativeai з підтримкою структурованих відповідей (response_schema) [6];
- в) Pydantic v2 — бібліотека валідації та серіалізації даних, що забезпечує жорстку типізацію всіх внутрішніх моделей та API-відповідей;
- г) DuckDuckGo Search — бібліотека для автономного пошуку в Інтернеті, що використовується модулем OSINT-верифікації для перевірки фактів без залежності від пропріетарних пошукових API.

Клієнтська частина побудована на стандартних веб-технологіях (JavaScript, HTML, CSS) відповідно до стандарту Chrome Extensions Manifest V3.

[Рисунок 2.1 — Діаграма компонентів системи VERITAS]

2.2 Мікроагентна архітектура аналітичного конвеєра

Центральним архітектурним рішенням системи є мікроагентна модель обробки даних. Замість використання єдиного пром프트у, що виконує увесь аналіз, систему побудовано як оркестрацію незалежних спеціалізованих мікроагентів, кожен з яких відповідає за окремий аспект аналізу.

2.2.1 Концепція незалежних мікроагентів

Кожен мікроагент є окремим викликом до мовної моделі із власним системним проммптом та жорстко типізованою схемою відповіді (Pydantic response_schema). Така декомпозиція забезпечує:

- а) ізоляцію помилок — хибний результат одного агента не впливає на роботу інших;
- б) спеціалізацію — кожен агент сфокусований на вузькій задачі, що підвищує якість аналізу;

в) незалежне тестування — кожен агент може бути протестований ізольовано.

Система включає чотири аналітичні агенти:

BiasAgent (агент об'єктивності) — аналізує текст на наявність емоційних маніпуляцій, токсичної лексики та суб'єктивних суджень автора. Повертає кількісні метрики: `toxic_words_count` (кількість токсичних слів), `opinion_sentences_count` (кількість суб'єктивних речень автора), `imbalance_detected` (прапорець дисбалансу думок). Критичне правило: агент не штрафує за емоційну лексику, якщо вона є частиною прямої цитати.

LogicAgent (агент логічного аналізу) — оцінює структурну цілісність аргументації та релевантність заголовку. Повертає: `clickbait_triggers_count` (кількість клікбейтних прийомів у заголовку), `headline_mismatch_severity` (ступінь невідповідності заголовку змісту, шкала 0–5), `logical_fallacies_count` (кількість логічних хиб).

FactAgent (агент фактуального вилучення) — витягує кількісні дані: `named_entities_count` (кількість іменованих сутностей), `text_citations` (список текстових цитат та джерел), `verifiable_claims` (перелік тверджень, які можна перевірити через OSINT, до 3 одиниць).

ValidationAgent (агент фінальної валідації, «Supreme Judge») — отримує результати всіх попередніх агентів та виконує роль контролю якості. Він відхиляє хибнопозитивні хайлайти (`false positives`), генерує фінальний текстовий пояснювач (`explainer`) та затверджує лише ті маркування, які дійсно свідчать про маніпуляцію.

[Рисунок 2.2 — Діаграма послідовності роботи мікроагентів]

2.2.2 Паралельне виконання

Для мінімізації часу відповіді системи три аналітичні агенти (BiasAgent, LogicAgent, FactAgent) запускаються конкурентно за допомогою механізму `asyncio.gather()` мови Python. Паралельно з AI-агентами виконуються також задачі вилучення гіперпосилань (`extract_links`) та перевірки репутації домену (`check_domain_reputation`) через OSINT-модуль. Таким чином, загальний час аналізу визначається найповільнішим із конкурентних потоків, а не їх сумою.

Після завершення паралельної фази послідовно виконуються:

а) OSINT-верифікація тверджень, виявлених FactAgent;

б) фінальна валідація через ValidationAgent;

в) детерміноване обчислення скорингу.

2.2.3 Принцип «Epistemic Humility»

Ключовим архітектурним обмеженням, вбудованим у системні промпти всіх агентів, є принцип «епістемічної скромності» (Epistemic Humility). Кожен AI-агент отримує інструкцію:

«Do NOT fact-check the article. You do not have internet access and are unaware of current events. Assume the factual premises of the article are true. Your ONLY job is to analyze [bias / logic / entities].»

Цей принцип забезпечує строге розділення відповідальностей: мовні моделі аналізують виключно стилістичні та структурні характеристики тексту, тоді як фактична верифікація делегується окремому модулю OSINT, що має реальний доступ до актуальних джерел в Інтернеті. Такий підхід запобігає поширеній проблемі LLM — генерації хибних фактичних суджень на основі застарілих тренувальних даних (так званих «галюцинацій»).

2.2.4 Chain of Thought

Для забезпечення прозорості та можливості аудиту результатів, кожен агент зобов'язаний заповнити поле `raw_thoughts` у своїй відповіді. Це поле містить покроковий опис міркувань агента перед формуванням кількісних метрик. Така техніка, відома як Chain of Thought (CoT), підвищує надійність результатів та дозволяє розробнику діагностувати причини некоректних оцінок.

2.3 Авторський алгоритм WMFA v5.1

Алгоритм WMFA (Weighted Multi-Factor Analysis) версії 5.1 є ядром скорингової підсистеми VERITAS. Його фундаментальний принцип — розділення відповідальності: AI-агенти повертають «сирі» лічильники (кількість токсичних слів, кількість логічних хиб тощо), а детерміновані функції Python обчислюють точні бали за математичними формулами. Це гарантує повну відтворюваність результатів: при однакових вхідних даних скоринг завжди поверне ідентичний результат.

Фінальний рейтинг довіри (Trust Score) обчислюється як зважене середнє п'яти незалежних критеріїв:

$$TS = 0.35 * S1 + 0.20 * S2 + 0.15 * S3 + 0.15 * S4 + 0.15 * S5 \quad (2.1)$$

де S1 — верифікація джерел, S2 — об'єктивність, S3 — релевантність заголовку, S4 — фактологічна насиченість, S5 — логічна послідовність. Кожен критерій оцінюється за шкалою від 0 до 100.

Розподіл ваг обґрунтовано емпірично на тестовій вибірці з 12 статей різної якості. Найбільшу вагу (35%) має верифікація джерел, оскільки наявність чи відсутність підтвердження фактів є визначальним індикатором достовірності.

Таблиця 2.1 — Критерії алгоритму WMFA v5.1 та їхні ваги

Критерій	Вага	Джерело даних
Верифікація джерел (S1)	0.35	Links + Citations + OSINT
Об'єктивність (S2)	0.20	BiasAgent
Релевантність заголовку (S3)	0.15	LogicAgent + Python
Фактологічна насиченість (S4)	0.15	FactAgent
Логічна послідовність (S5)	0.15	LogicAgent + BiasAgent

2.3.1 Критерій 1: Верифікація джерел (вага 35%)

Критерій оцінює рівень підтвердження тверджень статті зовнішніми джерелами. Оцінка формується з трьох компонентів.

Перший компонент — оцінка гіперпосилань. Кожне зовнішнє посилання у тексті класифікується за типом домену:

- а) довірені домени (reuters.com, bbc.com, .gov, .edu тощо) — бонус 25 балів за кожне;
- б) «сірі» домени (соціальні мережі, блоги, UGC-платформи) — 0 балів, оскільки вони не є верифікованими джерелами;
- в) інші зовнішні домени — бонус 5 балів;
- г) внутрішні посилання (на той самий домен) — не враховуються в позитивну частину оцінки.

Сумарна оцінка за посилання обмежена зверху 50 балами.

Другий компонент — оцінка текстових цитат. AI-агент витягує текстові згадки джерел (наприклад, «за словами міністра», «як повідомляє Reuters»). Кожна цитата,

що згадує авторитетну сутність з бази AUTHORITATIVE_ENTITIES (понад 50 організацій), отримує 30 балів; інші цитати — 10 балів.

Третій компонент — OSINT-репутаційний модифікатор. Якщо модуль перевірки репутації домену (п. 2.4.2) повернув індекс довіри (`reputation_index`, значення від 0 до 1), то застосовується лінійна корекція:

$$\text{delta_rep} = (\text{reputation_index} - 0.5) * 30 \quad (2.2)$$

Таким чином, домен з індексом 1.0 отримує бонус +15 балів, з індексом 0.0 — штраф -15 балів, а індекс 0.5 не впливає на оцінку.

Додатково реалізовано Anti-SEO механізм: якщо стаття містить понад 5 внутрішніх посилань, кожне наступне знижує оцінку на 2 бали (максимальний штраф — 20 балів). Це протидіє практиці SEO-накрутки, коли сайт масово посилається на власні сторінки для імітації джерельної бази.

2.3.2 Критерій 2: Об'єктивність (вага 20%)

Критерій оцінює рівень емоційних маніпуляцій у тексті. Для моделювання впливу токсичної лексики на сприйняття тексту було обрано функцію експоненціального спаду (*exponential decay*), що широко застосовується в теорії інформації та обробці сигналів:

$$S2 = 100 * \exp(-15 * d) \quad (2.3)$$

де d — нормалізована щільність маніпулятивних елементів:

$$d = (\text{toxic_count} + 0.5 * \text{opinion_count}) / \max(50, \text{word_count}) \quad (2.4)$$

Параметр `toxic_count` — це кількість токсичних/маніпулятивних слів, виявлених BiasAgent, а `opinion_count` — кількість суб'єктивних суджень автора (не цитат). Множник 0.5 для `opinion_count` відображає меншу шкідливість суб'єктивних суджень порівняно з відверто токсичною лексикою.

Обрано саме експоненціальну функцію, оскільки вона природно моделює ефект втрати довіри: перші кілька маніпулятивних слів завдають найбільшої шкоди сприйняттю (різке падіння оцінки), тоді як подальше зростання кількості лише незначно погіршує вже низьку оцінку. Коефіцієнт 15 підібрано емпірично для забезпечення оптимальної сепарації між нейтральними та маніпулятивними текстами.

Нормалізація за довжиною тексту (word_count у знаменнику) гарантує, що 5 токсичних слів у статті з 2000 слів не будуть оцінені так само критично, як 5 токсичних слів у замітці зі 100 слів. Мінімальне значення знаменника (50) запобігає похибці при дуже коротких текстах.

2.3.3 Критерій 3: Релевантність заголовку (вага 15%)

Критерій виявляє клікбейт та невідповідність заголовку змісту статті. Оцінка формується геометричною прогресією за ступенем невідповідності:

$$S3 = 100 * 0.75^{\text{severity} - \text{triggers}} * 5 \quad (2.5)$$

де severity — рівень невідповідності заголовку змісту (0–5), визначений LogicAgent, а triggers — загальна кількість клікбейтних прийомів.

Використання множника 0.75 (замість лінійного штрафу) моделює мультиплікативний ефект дисконтування довіри: кожен наступний рівень невідповідності знижує залишок довіри на 25%. При $\text{severity} = 1$ оцінка складає 75, при $\text{severity} = 2$ — 56.25, при $\text{severity} = 5$ — 23.73.

Додатково Python-модуль виконує власну детекцію клікбейту, незалежну від AI:

а) CAPSLOCK-детекція: якщо понад 30% літер заголовку написано великими літерами, це вважається додатковим клікбейт-тригером;

б) словникова детекція: заголовок перевіряється на наявність 15+ клікбейтних ключових слів українською та англійською мовами («шок», «терміново», «сенсація», «shocking», «urgent» тощо).

2.3.4 Критерій 4: Фактологічна насиченість (вага 15%)

Критерій оцінює інформативність статті через щільність іменованих сутностей (імена, дати, географічні назви, грошові суми), витягнутих FactAgent. Для формування оцінки використано функцію насичення, відому як рівняння Міхаеліса-Ментен, що описує кінетику ферментативних реакцій у біохімії та моделює закон спадної граничної корисності в економіці:

$$S4 = 5 + 95 * d / (d + 0.02) \quad (2.6)$$

де $d = \text{entity_count} / \max(50, \text{word_count})$ — нормалізована щільність сутностей.

Ця функція має кілька важливих властивостей:

- а) базовий рівень (5 балів) — навіть текст без жодної іменованої сутності отримує мінімальну оцінку, оскільки відсутність фактів не означає маніпулятивності;
- б) асимптотичне наближення до 100 — при збільшенні щільності сутностей оцінка зростає, але ніколи не перевищує 100. Це відображає ефект «інформаційного насичення»: після певного порогу додаткові факти не підвищують якість тексту;
- в) швидке зростання на початку — перші факти різко підвищують оцінку, що відповідає інтуїтивному розумінню якості журналістики.

Константа 0.02 визначає точку напівнасичення (half-saturation) та підібрана емпірично.

2.3.5 Критерій 5: Логічна послідовність (вага 15%)

Критерій оцінює структурну цілісність аргументації на основі кількості логічних хиб (ad hominem, false causality, strawman тощо), виявлених LogicAgent. Використано модель мультиплікативної деградації з теорії надійності послідовних систем:

$$S5 = 100 * 0.80^{\text{fallacy_count}} \quad (2.7)$$

Кожна виявлена логічна хиба «дисконтує» залишок довіри до аргументації на 20%. При $\text{fallacy_count} = 1$ оцінка складає 80, при $\text{fallacy_count} = 2$ — 64, при $\text{fallacy_count} = 5$ — 32.77.

Якщо додатково BiasAgent визначив $\text{imbalance_detected} = \text{True}$ (стаття на контроверсійну тему повністю ігнорує одну зі сторін), застосовується додатковий множник 0.8:

$$S5' = S5 * 0.8 \quad (2.8)$$

Використання мультиплікативної (а не адитивної) схеми відображає принцип «ланцюга аргументації»: кожна наступна логічна хиба підриває довіру до всієї побудови загалом.

2.3.6 Фінальна агрегація та захисні механізми

Після обчислення всіх п'яти критеріїв виконується фінальна агрегація за формулою (2.1) з додатковими захисними механізмами.

Harmonic Drag. Якщо мінімальний із п'яти критеріїв менший за 25 балів, застосовується додаткове зниження фінальної оцінки:

$$\text{delta_drag} = (25 - \min(S1 \dots S5)) * 0.15 \text{ (2.9)}$$

Максимальне значення drag складає 3.75 бала. Цей механізм запобігає ситуації, коли стаття з одним критично слабким аспектом отримує незаслужено високу загальну оцінку за рахунок сильних показників у інших критеріях.

Critical OSINT Penalty. Якщо OSINT-модуль виявив, що одне або більше тверджень статті прямо суперечить інформації з авторитетних зовнішніх джерел (`verdict = CONTRADICTED`), застосовується штраф -40 балів:

$$\text{TS_final} = \text{TS} - 40 \text{ (2.10)}$$

Цей штраф є найжорсткішим механізмом системи та спрацьовує лише при доведеному факті дезінформації, підтвердженому незалежним пошуком. Одночасно з критичним штрафом обмежуються значення індивідуальних критеріїв (не вище 60), щоб інтерфейс користувача не показував «відмінну об'єктивність» для доведеного фейку.

Score Clamping. Фінальний результат завжди обмежується діапазоном [0, 100].

2.4 Модуль OSINT-верифікації

Модуль OSINT (Open Source Intelligence) реалізує автономну перевірку фактів через пошук у відкритих джерелах Інтернету. Архітектурно він побудований як RAG-конвеєр (Retrieval-Augmented Generation).

2.4.1 Архітектура RAG-конвеєра

Конвеєр верифікації складається з трьох послідовних етапів.

Етап 1: Генерація пошукових запитів (QueryGeneratorAgent). FactAgent повертає до 3 верифікованих тверджень (`verifiable_claims`) — конкретних фактичних стверджень, які можна перевірити. QueryGeneratorAgent за допомогою Gemini перекладає кожне твердження на англійську мову та формулює пошуковий запит для міжнародних джерел. Крос-лінгвальна трансляція (UK-EN) забезпечує доступ до глобальної англомовної бази знань.

Етап 2: Пошук доказів (DuckDuckGo Search). Для кожного запиту виконується пошук через бібліотеку `duckduckgo-search` з параметрами:

а) `max_results = 5` — обмеження кількості результатів;

б) `timeout = 10` секунд — захист від зависання;

в) region = "wt-wt" — глобальний пошук без географічного фільтру.

Результатом є набір текстових фрагментів (snippets) з посиланнями на першоджерела.

Етап 3: Семантичне порівняння (CrossReferenceAgent). Мовна модель порівнює оригінальне твердження зі статті з отриманими фрагментами та визначає вердикт:

- а) CONFIRMED — знайдені докази підтверджують твердження;
- б) CONTRADICTED — знайдені докази прямо суперечать твердженню;
- в) UNVERIFIED — недостатньо доказів для висновку.

При вердикті CONTRADICTED створюється хайлайт з категорією «Спростовано OSINT-пошуком», що активує Critical Penalty (формула 2.10).

[Рисунок 2.3 — Схема RAG-конвеєра OSINT-верифікації]

2.4.2 Перевірка репутації домену

Паралельно з аналізом контенту виконується OSINT-перевірка репутації домену-публікатора. DomainReputationAgent формує пошуковий запит вигляду «[domain] media reliability credibility rating» та аналізує знайдені фрагменти з медіа-спостерігачів та організацій з моніторингу ЗМІ.

Агент повертає структуру з трьома полями:

- а) trust_index (від 0 до 1) — числовий індекс довіри;
- б) classification — категорія: Trusted / Gray / Unknown / Disreputable;
- в) background_summary — текстовий опис знахідок.

Значення trust_index використовується формулою (2.2) як OSINT-репутаційний модифікатор у критерії верифікації джерел.

Для запобігання хибнопозитивним результатам домени попередньо перевіряються за статичними списками: TRUSTED_DOMAINS (понад 80 доменів авторитетних медіа та державних установ) та GRAY_DOMAINS (понад 30 доменів соціальних мереж та блог-платформ). OSINT-запит виконується лише для доменів, яких немає в жодному зі списків.

2.4.3 Стратегія вердикту

Система використовує консервативну стратегію вердиктів. Критичний штраф (-40 балів) застосовується виключно при вердикті CONTRADICTED, тобто лише тоді, коли знайдено конкретні докази, що спростовують твердження статті. Вердикт UNVERIFIED не штрафує статтю, оскільки відсутність підтвердження у пошуку ще не означає хибність інформації — твердження може стосуватися нещодавньої або локальної події, ще не проіндексованої пошуковими системами.

2.5 Клієнтська частина: Chrome Extension

Клієнтська частина реалізована як браузерне розширення для Google Chrome відповідно до стандарту Manifest V3 — найновішого стандарту розробки розширень, що забезпечує покращену безпеку, продуктивність та конфіденційність [10].

2.5.1 Маніфест та дозволи

Маніфест розширення (manifest.json) визначає набір необхідних дозволів:

- а) activeTab — доступ до поточної активної вкладки для вилучення тексту;
- б) storage — доступ до chrome.storage.local для кешування результатів та налаштувань;
- в) host_permissions: http://127.0.0.1:8000/* — дозвіл на HTTP-запити до локального серверу.

Розширення складається з трьох ключових компонентів: Background Service Worker, Content Script та Popup UI.

2.5.2 Екстракція тексту за допомогою Readability.js

Для вилучення чистого тексту статті зі сторінки використовується бібліотека Mozilla Readability.js [11]. Перед викликом Readability Content Script виконує попереднє очищення DOM від шумових елементів (навігація, реклама, бічні панелі) за допомогою переліку CSS-селекторів.

Після вилучення тексту Content Script індексує кожен параграф, присвоюючи йому унікальний атрибут data-veritas-id. Цей атрибут є ключовим для подальшого зворотного зв'язку: коли сервер повертає хайлайт з вказанням номера параграфа, Content Script може миттєво знайти відповідний DOM-елемент за $O(1)$ складністю та візуально позначити його.

2.5.3 Background Service Worker

Background Service Worker є центральним координатором процесу аналізу. Він виконує кілька ключових функцій.

Стійкість до закриття popup. В архітектурі Chrome Extensions Manifest V3 popup-вікно закривається при кожному кліку поза ним, знищуючи весь стан JavaScript. Background Service Worker продовжує роботу незалежно від стану popup, зберігаючи стан аналізу та результати.

Машина станів. Для кожної вкладки підтримується стан аналізу з наступними переходами: Initial - Analyzing - Complete | Error. Popup при відкритті запитує поточний стан через `chrome.runtime.sendMessage` та відображає відповідний інтерфейс.

Кешування результатів. Результати аналізу зберігаються у `chrome.storage.local` з ключем, що дорівнює URL сторінки, та TTL = 1 година. Це дозволяє миттєво відображати результати при повторному відкритті тієї самої сторінки.

2.5.4 Popup UI

Popup-вікно розширення реалізує систему станів із п'ятьма візуальними екранами: Initial (початковий), Analyzing (процес аналізу), Results (результати), Error (помилка), Settings (налаштування).

Екран Results є основним для взаємодії користувача та містить:

- а) анімований SVG Score Ring — кругова діаграма, що візуалізує загальний рейтинг довіри. Анімація реалізована через `requestAnimationFrame` з функцією `ease-out cubic` для плавного заповнення кільця від 0 до цільового значення;
- б) SVG Pentagon Radar Chart — п'ятикутна діаграма, що відображає значення кожного з п'яти критеріїв. Реалізована без використання зовнішніх бібліотек через динамічне створення SVG-елементів (`polygon`, `circle`, `text`) у JavaScript;
- в) Highlight Navigator — компонент навігації по знайдених проблемних фрагментах. Відображає кількість попереджень (`warnings`) та високих ризиків (`risks`) та дозволяє послідовно переходити між ними.

[Рисунок 2.4 — Інтерфейс Popup UI у стані Results]

2.5.5 Content Script: система підсвічування DOM

Після отримання результатів аналізу від серверу Content Script ін'єктує візуальне маркування безпосередньо у DOM-структуру веб-сторінки. Кожен проблемний параграф обгортається елементом з CSS-класом, що відповідає рівню серйозності:

- а) warning (жовтий фон) — сумнівне твердження або легка маніпуляція;
- б) risk (червоний фон) — високий ризик: доведений фейк або грубе викривлення.

При наведенні курсору на маркований фрагмент відображається tooltip з поясненням причини маркування. Tooltip реалізований як абсолютно позиціонований DOM-елемент з динамічним розрахунком координат для уникнення виходу за межі viewport.

Для навігації між хайлайтами з PopUp використовується функція `scrollIntoView` з пульсуючою CSS-анімацією, що привертає увагу користувача до поточного маркування.

[Рисунок 2.5 — Приклад підсвічування сумнівних фрагментів на веб-сторінці]

2.6 Системи кешування та локалізації

2.6.1 Дворівневе кешування

Система реалізує кешування на двох рівнях для мінімізації повторних запитів до AI-моделей.

Backend-кеш. Реалізований як in-memory словник з ключем, що обчислюється як SHA-256 хеш від конкатенації домену, заголовку та мови аналізу. TTL = 12 годин. При отриманні запиту сервер спочатку перевіряє наявність валідного кешу та повертає збережений результат без повторного аналізу.

Extension-кеш. Реалізований через `chrome.storage.local` з ключем, що дорівнює URL сторінки, та TTL = 1 година. PopUp при відкритті спочатку перевіряє локальний кеш, і лише при його відсутності або закінченні TTL ініціює новий запит.

2.6.2 Двомовна підтримка

Система підтримує українську (uk) та англійську (en) мови інтерфейсу. Локалізація реалізована через механізм `data-i18n` атрибутів: кожен текстовий елемент інтерфейсу має атрибут `data-i18n` з ключем перекладу, а функція `applyTranslations()` підставляє відповідний текст зі словника `translations`.

Мова обирається користувачем у розділі Settings та зберігається у `chrome.storage.local`. Параметр мови також передається на сервер у запиті аналізу, що дозволяє генерувати текстовий пояснювач (explainer) відповідною мовою.